MOTILAL NEHRU NATIONAL INSTITUTE OF TECHNOLOGY ALLAHABAD
PRAYAGRAJ, INDIA – 211004

A PROJECT REPORT ON

Insurance & Claims Management System

with ML-based Fraud Risk Scoring

PROJECT Number

E6

Submitted By

Satyam Kumar (Reg. No: 20243587)
Shashank Lakkarsu (Reg. No: 20243547)
Rajiv Thodla (Reg. No: 20243539)
Shabbeer Mohammed (Reg. No: 20243544)

Submitted as DBMS Project Work

Bachelor of Technology (B. Tech.)
Department of Computer Science and Engineering

MOTILAL NEHRU NATIONAL INSTITUTE OF TECHNOLOGY ALLAHABAD
PRAYAGRAJ, UTTAR PRADESH – 211004

2026–2027

Table of Contents

1. Abstract 3
2. Introduction and Problem Statement 3
2.1 Background 3
2.2 Problem statement 3
3. Project Objectives and Scope 4
3.1 Objectives 4
3.2 Scope 4
4. System Analysis and Requirements 4
4.1 Existing system and its limitations 4
4.2 Proposed system 4
4.3 Functional requirements 5
4.4 Business rules 5
4.5 Non-functional requirements 6
4.6 Hardware and software requirements 6
5. ER/EER Diagram and Database Schema 6
5.1 Entities and relationships 6
5.2 Relational schema 7
6. Keys, Functional Dependencies and Normalization 8
6.1 Keys 8
6.2 Normalization process 8
6.3 Functional dependencies and normal form of each table 9
7. SQL Implementation 9
7.1 DDL and DML 10
7.2 Joins 10
7.3 Subqueries 10
7.4 Aggregation and ROLLUP 11
7.5 Window functions, CTEs and set operations 11
8. Views, Indexes, Triggers and Transactions 12
8.1 Views 12
8.2 Indexes 12
8.3 Triggers 13
8.4 Stored procedures and functions 13
8.5 Transactions and concurrency control 14
9. Front-End Design and Database Connectivity 14
9.1 Front-end design 14
9.2 Database connectivity 15

9.3 Security 16
10. CRUD Operations and Report Generation 17
10.1 Generic CRUD 17
10.2 Workflow operations 18
10.3 Reports 18
11. AI/ML Implementation and Model Evaluation 19
11.1 Problem definition 19
11.2 Data and features 20
11.3 Training method 20
11.4 Evaluation on the held-out test set 21
11.5 Deployment in the application 21
11.6 Limitations 22
12. Testing, Results and Screenshots 22
12.1 Testing approach 22
12.2 Automated test results 22
12.3 Business-rule demonstrations 23
12.4 Results summary 24
13. Conclusion and Future Scope 24
13.1 Conclusion 24
13.2 Future scope 25
14. References 25

1. Abstract

Insurance companies handle a large volume of policies, premium instalments and claims, and a small share of those claims are fraudulent. This project builds an **Insurance & Claims Management System** on **MySQL 8.0** that keeps this data consistent and helps staff decide which claims need a closer look.

The database has **15 tables in Boyce-Codd Normal Form**, connected by primary, foreign, unique and CHECK constraints. Ten business rules (BR1–BR10), such as "a claim cannot exceed the sum assured" or "a payout must not exceed the approved amount", are enforced inside the database by **12 triggers**. The claim workflow (submit → review → approve/reject → pay) runs through **16 stored procedures** that use explicit transactions, row-level locks (`SELECT … FOR UPDATE`) and rollback on error. **10 views** feed the user interface and the reports, and **7 secondary indexes** support the main filters and searches; the effect of four of them was measured with `EXPLAIN ANALYZE`.

A **FastAPI** (Python) back end exposes the database through a REST API, and a plain HTML/JavaScript front end gives staff a dashboard, generic CRUD screens for every master table, a claims work queue and eight downloadable reports. A **Random Forest** classifier trained with scikit-learn on 2,353 resolved claims scores every open claim for fraud risk. On a held-out test set it reached a ROC-AUC of **0.948**, precision **0.82** and recall **0.74**. The model's decision threshold is stored in the database, so the LOW / MEDIUM / HIGH risk bands in the views always follow the active model.

The system was loaded with a synthetic but realistic dataset (1,600 customers, 3,500 policies, 10,484 premium instalments and 2,455 claims) and verified by an automated test suite of **30 tests (all passing)**, 21 deliberate rule-violation demonstrations and a concurrency demonstration that shows row locking and deadlock detection.

2. Introduction and Problem Statement

2.1 Background

An insurance policy is a contract: the customer (policy holder) pays a **premium**, usually in instalments, and in return the insurer promises to pay up to the **sum assured** if an insured event happens during the policy period. When something does happen, the customer files a **claim**. A claims **adjuster** reviews the claim and its documents, approves an amount (or rejects it), and the insurer then pays the claim, often directly to a hospital or garage (the **provider**).

These steps involve many related records: customers, agents who sold the policy, products, policies, nominees, instalments, claims, documents, assessments and payouts. They have to stay consistent with each other. For example, a payout must never be larger than what the adjuster approved, and a claim must never be filed on a policy that has lapsed for non-payment.

2.2 Problem statement

Small and mid-sized insurers and third-party administrators often track this work in spreadsheets or in loosely connected applications. That leads to three recurring problems:

- **Inconsistent data:** duplicated customer addresses, claims attached to cancelled policies, payouts larger than the approved amount, and no reliable history of who changed a claim's status.
- **Rules enforced only by people:** checks such as "the adjuster may only approve up to their limit" depend on staff remembering them, and two staff members can act on the same claim at the same time.
- **Fraud found too late:** suspicious claims (filed long after the incident, with few documents, at a non-network provider) are only noticed after the money has been paid.

The goal of this project is to design and implement a normalized relational database that enforces these business rules itself, keeps the claim workflow correct even under concurrent use, gives staff a simple web interface with CRUD screens and reports, and adds a machine-learning model that ranks open claims by fraud risk so that adjusters look at the riskiest ones first.

3. Project Objectives and Scope

3.1 Objectives

1. Design an ER model for the insurance domain and convert it into a relational schema in **BCNF**.
2. Enforce data integrity with **PK, FK, UNIQUE and CHECK constraints** and enforce the ten business rules (BR1–BR10) with **triggers**, so that no client can bypass them.
3. Implement the policy and claim life-cycles as **stored procedures with transactions**, including an atomic "approve and pay" operation and safe behaviour under **concurrent access**.
4. Demonstrate the full range of SQL: DDL, DML, all join types, subqueries, aggregation with ROLLUP, window functions, recursive CTEs and set operations.
5. Create **views** for the application and reports, and **indexes** justified by measured query plans.
6. Build a **web front end** connected to the database through a REST API, with CRUD for every master table, a claims work queue and downloadable reports.
7. Train, evaluate and deploy a **fraud-risk model** that scores open claims and stores its score in the database.
8. Verify everything with automated **tests** and saved demonstration outputs.

3.2 Scope

The system is an internal tool for insurer staff and covers:

- Master data: customers, agents, products, providers, adjusters and pincodes.
- Policies: selling, nominees, premium instalments, payment, lapse, re-activation and expiry.
- Claims: filing, documents, review, approval or rejection, payout and an audit trail.
- A dashboard and eight reports with CSV export.
- Fraud-risk scoring of open claims.

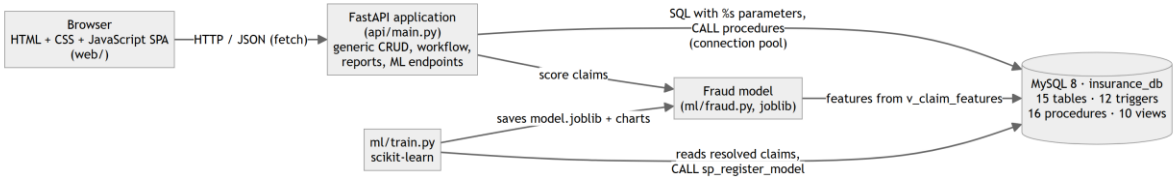
4. System Analysis and Requirements

4.1 Existing system and its limitations

In the manual or spreadsheet-based process, each department keeps its own copy of customer and policy data, status changes are not recorded, validation is done by hand and reports are built manually each month. There is no locking, so two people can settle the same claim twice, and there is no systematic way to flag risky claims.

4.2 Proposed system

The proposed system keeps one normalized database as the single source of truth. All rules live in the database (constraints, triggers, procedures), so they apply whether data arrives from the web UI, the API, a script or the MySQL command line. A thin API layer translates HTTP requests into parameterised SQL and procedure calls, and the front end only displays data and sends user actions.



System architecture

4.3 Functional requirements

Functional requirements

ID	Requirement	Implemented by
F1	Create, read, update, delete and search master data (customers, agents, products, providers, adjusters, pincodes, nominees, documents)	Generic CRUD API + CRUD screens
F2	Sell a policy: validate cover against the product, compute the premium, assign a policy number, raise the first instalment	`sp_sell_policy`, `trg_policy_bi`, `fn_annual_premium`
F3	Premium life-cycle: generate instalments, record payments, lapse overdue policies, re-activate on payment, expire policies	`sp_generate_instalments`, `sp_pay_premium`, `sp_lapse_overdue_policies`, `sp_expire_policies`, `trg_payment_au`
F4	File a claim with documents	`sp_file_claim`, `trg_claim_bi`
F5	Claim workflow: start review, request info, approve, reject, pay	`sp_start_review`, `sp_request_info`, `sp_approve_claim`, `sp_reject_claim`, `sp_pay_claim`
F6	Approve and pay as one atomic operation	`sp_approve_and_pay_claim`
F7	Score open claims for fraud risk and show a risk band	`ml/train.py`, `ml/fraud.py`, `fraud_model`, `v_claim_summary`
F8	Dashboard and eight reports, downloadable as CSV	`api/reports.py` over the views

4.4 Business rules

Business rules enforced by the database

Rule	Description	Enforced in
BR1	A claim cannot be filed on a LAPSED or CANCELLED policy	`trg_claim_bi`
BR2	The incident date must lie inside the policy period	`trg_claim_bi`
BR3	The claimed amount cannot exceed the sum assured	`trg_claim_bi`
BR4	Claim status may only move SUBMITTED → UNDER_REVIEW → APPROVED/REJECTED, APPROVED → PAID; review needs an adjuster, a decision needs an assessment, PAID needs a payout; claim details freeze after submission	`trg_claim_bu`
BR5	An approved amount cannot exceed the adjuster's approval limit or the claimed amount	`trg_assessment_bi`
BR6	Only an APPROVED claim can be paid; payout ≤ approved amount; total payouts on a policy ≤ sum assured	`trg_payout_bi`
BR7	Recording a payout automatically marks the claim PAID	`trg_payout_ai`

Rule	Description	Enforced in
BR8	Nominee shares on a policy cannot exceed 100%	`trg_nominee_ai`, `trg_nominee_au`
BR9	Paying the last overdue instalment re-activates a LAPSED policy	`trg_payment_ai`, `trg_payment_au`
BR10	Every claim status change is written to an audit log	`trg_claim_ai`, `trg_claim_au`

4.5 Non-functional requirements

- **Integrity:** rules are enforced in the database, not only in the UI.
- **Atomicity and concurrency:** multi-step operations either complete fully or leave no trace; two users acting on the same claim cannot both succeed.
- **Security:** all user input reaches SQL only as bound parameters; table and column names come from a fixed whitelist; the application connects with its own MySQL user, not root.
- **Performance:** list screens and reports respond in well under a second on the full dataset; indexes are justified with `EXPLAIN ANALYZE`.
- **Usability:** a plain, table-oriented interface similar to typical database administration tools.

4.6 Hardware and software requirements

Software and hardware used

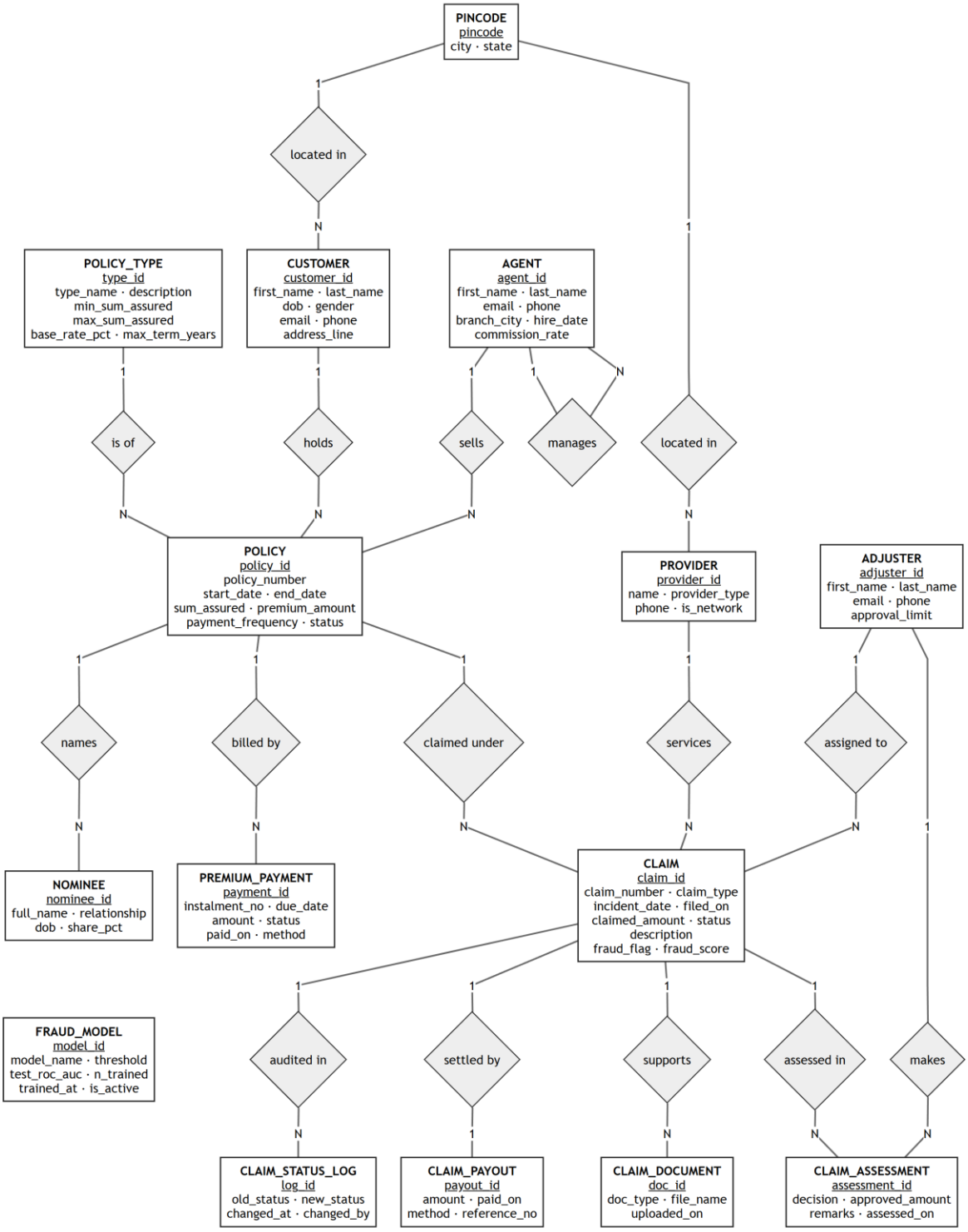
Component	Used in this project
Operating system	Windows 11
Database	MySQL Community Server 8.0.46 (InnoDB engine, utf8mb4, REPEATABLE READ isolation)
Back end	Python 3.12, FastAPI, Uvicorn, mysql-connector-python (connection pool)
Front end	HTML5, CSS, plain JavaScript (single-page application, no framework)
Machine learning	scikit-learn, pandas, NumPy, matplotlib, joblib
Testing	pytest with FastAPI TestClient against a separate `insurance_test` database
Tools	MySQL Workbench / mysql CLI, Visual Studio Code, Google Chrome
Hardware (minimum)	Any PC with 4 GB RAM and about 500 MB free disk space

5. ER/EER Diagram and Database Schema

5.1 Entities and relationships

The main entities are **Customer**, **Agent**, **Policy Type** (product), **Policy**, **Nominee**, **Premium Payment**, **Claim**, **Provider**, **Adjuster**, **Claim Document**, **Claim Assessment**, **Claim Payout** and **Claim Status Log**. **Pincode** was separated out during normalization, and **Fraud Model** stores the trained ML models.

In the diagram, rectangles are entities with their attributes (the primary key is underlined), diamonds are relationships, and the 1 and N on each line give the cardinality. Every relationship is one-to-many, except that a claim is settled by at most one payout (1 : 1) and an agent manages other agents (a recursive 1 : N).



ER diagram

5.2 Relational schema

Each entity becomes one table, and each 1 : N relationship becomes a foreign key on the N side.

Tables and keys

Table	Primary key	Foreign keys	Purpose	Rows
pincode	pincode	—	Lookup of pincode → city, state	450

Table	Primary key	Foreign keys	Purpose	Rows
agent	agent_id	manager_id → agent	Agents who sell policies; self-reference gives the hierarchy	40
customer	customer_id	pincode → pincode	Policy holders	1,600
policy_type	type_id	—	Product catalogue (Health, Motor, Life, Home, Travel) with cover limits and base rate	5
provider	provider_id	pincode → pincode	Hospitals, clinics and garages; network / non-network	120
adjuster	adjuster_id	—	Claim assessors with an approval limit	15
policy	policy_id	customer_id, type_id, agent_id	One contract for one product	3,500
nominee	nominee_id	policy_id	Beneficiaries and their share	2,185
premium_payment	payment_id	policy_id	One row per instalment (due, paid, failed)	10,484
claim	claim_id	policy_id, provider_id, adjuster_id	A claim with status, fraud flag (label) and fraud score	2,455
claim_document	doc_id	claim_id	Supporting document metadata	7,252
claim_assessment	assessment_id	claim_id, adjuster_id	Adjuster decisions: APPROVED, REJECTED, NEEDS_INFO	2,484
claim_payout	payout_id	claim_id (unique)	At most one settlement per claim	1,882
claim_status_log	log_id	claim_id	Audit trail written only by triggers	9,101
fraud_model	model_id	—	Registry of trained models; exactly one is active	1

6. Keys, Functional Dependencies and Normalization

6.1 Keys

- **Primary keys:** one per table (surrogate integers, plus the natural `pincode`).
- **Candidate (alternate) keys, declared UNIQUE:** `customer.email`, `agent.email`, `adjuster.email`, `policy_type.type_name`, `policy.policy_number`, `claim.claim_number`, `claim_payout.reference_no`, `claim_payout.claim_id`, and the composite key `(policy_id, instalment_no)` in `premium_payment`.
- **Foreign keys:** 16 in total, each with an explicit ON DELETE / ON UPDATE action.
- **CHECK constraints:** 32 domain checks, for example `pincode REGEXP '^[0-9]{6}$'`, `end_date > start_date`, `share_pct BETWEEN 0 AND 100`, and the cross-column rule `(status = 'PAID') = (paid_on IS NOT NULL)` on instalments.

6.2 Normalization process

We started from the single "policy register" that an insurer would keep in a spreadsheet, with one row per policy instalment:

```
POLICY_REGISTER(policy_number, customer_name, customer_email, address, pincode, city, state,
                product, base_rate, max_term, agent_name, agent_branch, start_date, end_date,
                sum_assured, nominees{name, relation, share}, instalment_no, due_date, amount, paid_on)
```

- **1NF:** the repeating group `nominees{…}` violates atomicity. It was moved to its own relation **NOMINEE(policy, full_name, relationship, share_pct)**. Every attribute is now atomic.

- **2NF:** the key of the register is (policy_number, instalment_no), but customer, product and policy attributes depend on `policy_number` alone (partial dependencies). Splitting gives **POLICY** (one row per policy) and **PREMIUM_PAYMENT(policy, instalment_no, due_date, amount, status, paid_on)**.
- **3NF:** inside POLICY there are transitive dependencies: policy → customer_email → name, address; policy → product → base_rate, max_term; policy → agent → branch. These became **CUSTOMER**, **POLICY_TYPE** and **AGENT**, referenced by foreign keys.
- **BCNF:** in CUSTOMER (and PROVIDER), `pincode → city, state` still holds and `pincode` is not a key of CUSTOMER. Moving it to **PINCODE(pincode, city, state)** removes the last dependency whose left side is not a superkey. The same analysis was applied to the claim side (claim, document, assessment, payout, status log).

6.3 Functional dependencies and normal form of each table

For every table, the only non-trivial functional dependencies have a candidate key on the left-hand side, so every table is in **BCNF** (and therefore also in 3NF, 2NF and 1NF).

Functional dependencies and normal form

Table	Non-trivial functional dependencies	NF
pincode	pincode → city, state	BCNF
agent	agent_id → all attributes; email → all attributes	BCNF
customer	customer_id → all; email → all (pincode → city, state moved out)	BCNF
policy_type	type_id → all; type_name → all	BCNF
provider	provider_id → name, provider_type, pincode, phone, is_network	BCNF
adjuster	adjuster_id → all; email → all	BCNF
policy	policy_id → all; policy_number → all	BCNF
nominee	nominee_id → policy_id, full_name, relationship, dob, share_pct	BCNF
premium_payment	payment_id → all; (policy_id, instalment_no) → all	BCNF
claim	claim_id → all; claim_number → all	BCNF
claim_document	doc_id → claim_id, doc_type, file_name, uploaded_on	BCNF
claim_assessment	assessment_id → claim_id, adjuster_id, decision, approved_amount, remarks, assessed_on	BCNF
claim_payout	payout_id → all; claim_id → all; reference_no → all	BCNF
claim_status_log	log_id → claim_id, old_status, new_status, changed_at, changed_by	BCNF
fraud_model	model_id → model_name, threshold, test_roc_auc, n_trained, trained_at, is_active	BCNF

7. SQL Implementation

All SQL lives in numbered script files under `db/`, so the whole database can be rebuilt from scratch with one command (`python scripts/build_db.py`, or `db/run_all.sql` from the mysql client).

SQL script files

File	Contents
`01_schema.sql`	DDL: 15 tables with all constraints
`02_triggers.sql`	12 triggers (BR1–BR10, product cover check)
`03_procedures.sql`	3 functions and 16 stored procedures
`04_views.sql`	10 views

File	Contents
`05_seed_data.sql`	Generated dataset (1,600 customers, 3,500 policies, 2,455 claims, …)
`06_indexes.sql`	7 secondary indexes
`07_queries.sql`	DML, joins, subqueries, aggregation, window functions, CTEs, set operations, reports
`08_rule_demos.sql`	Every business rule and transaction demonstrated, including the expected errors

7.1 DDL and DML

Tables are created with `CREATE TABLE … ENGINE=InnoDB`, which gives foreign keys, transactions and row-level locking. The seed data is loaded with multi-row `INSERT` statements. Claims in the seed are not inserted with their final status: they are replayed through the same procedures the application uses (file → review → approve/reject → pay), so every seeded claim has passed every trigger. The query script also runs an `INSERT`, an `UPDATE` with a join and a `DELETE` inside a transaction that is then rolled back.

7.2 Joins

Inner join across five tables

```
SELECT cl.claim_number, CONCAT(c.first_name, ' ', c.last_name) AS customer, pt.type_name,
       pr.name AS provider, cl.claimed_amount, po.amount AS paid, DATE(po.paid_on) AS paid_on
  FROM claim cl
  JOIN policy p        ON p.policy_id   = cl.policy_id
  JOIN customer c      ON c.customer_id = p.customer_id
  JOIN policy_type pt  ON pt.type_id    = p.type_id
  JOIN claim_payout po ON po.claim_id   = cl.claim_id
  LEFT JOIN provider pr ON pr.provider_id = cl.provider_id
 ORDER BY po.paid_on DESC
 LIMIT 10
```

Output:

```
+-----------------+--------------+-----------+--------------------------+----------------+-----------+------------+
| claim_number    | customer     | type_name | provider                 | claimed_amount | paid      | paid_on    |
+-----------------+--------------+-----------+--------------------------+----------------+-----------+------------+
| CLM-2026-002406 | Neha Tiwari  | Travel    | Care Hospital, Ahmedabad |       49500.00 |  49500.00 | 2026-11-04 |
| CLM-2026-002381 | Ritika Reddy | Motor     | Speed Motors, Ahmedabad  |        3300.00 |   3300.00 | 2026-11-03 |
| CLM-2026-002353 | Meera Shah   | Motor     | Prime Motors, Prayagraj  |       25100.00 |  25100.00 | 2026-11-02 |
| CLM-2026-002352 | Kavya Kapoor | Health    | Apex Hospital, Jaipur    |        2600.00 |   1600.00 | 2026-11-02 |
|  ...
+-----------------+--------------+-----------+--------------------------+----------------+-----------+------------+
10 rows in set (0.03 sec)
```

The script also has a left anti-join (181 customers hold no policy), a self join that lists each agent with their manager, and a full outer join, emulated as a left join UNION a right join because MySQL has no `FULL OUTER JOIN`.

7.3 Subqueries

Correlated subquery: claims more than 3× their type's average

```
SELECT cl.claim_number, cl.claim_type, cl.claimed_amount
  FROM claim cl
 WHERE cl.claimed_amount > 3 * (SELECT AVG(c2.claimed_amount) FROM claim c2
                                WHERE c2.claim_type = cl.claim_type)
 ORDER BY cl.claimed_amount DESC LIMIT 10
```

Output:

```
+-----------------+------------------+----------------+
| claim_number    | claim_type       | claimed_amount |
```

```
+-----------------+------------------+----------------+
| CLM-2025-001226 | CRITICAL_ILLNESS |     2693800.00 |
| CLM-2024-000324 | CRITICAL_ILLNESS |     2434000.00 |
| CLM-2026-001817 | CRITICAL_ILLNESS |     1538800.00 |
| CLM-2023-000064 | CRITICAL_ILLNESS |     1067900.00 |
|  ...
+-----------------+------------------+----------------+
10 rows in set (3.78 sec)
```

Other subqueries: a scalar subquery (385 claims are above the average of ₹66,607), `IN` (customers with a rejected claim), `NOT EXISTS` (1,163 active policies never had a claim) and `> ALL` (providers paid more than every non-network provider).

7.4 Aggregation and ROLLUP

GROUP BY … WITH ROLLUP

```
SELECT IF(GROUPING(pt.type_name), 'ALL PRODUCTS', pt.type_name) AS product,
       IF(GROUPING(p.status), 'all statuses', p.status)          AS status,
       COUNT(*) AS policies, SUM(p.sum_assured) AS total_cover
  FROM policy p JOIN policy_type pt ON pt.type_id = p.type_id
 GROUP BY pt.type_name, p.status WITH ROLLUP
```

Output:

```
+--------------+--------------+----------+---------------+
| product      | status       | policies | total_cover   |
+--------------+--------------+----------+---------------+
| Health       | ACTIVE       |      460 |  445900000.00 |
| Health       | CANCELLED    |       12 |   11300000.00 |
| Health       | EXPIRED      |      673 |  632800000.00 |
| Health       | LAPSED       |       27 |   22600000.00 |
| Health       | all statuses |     1172 | 1112600000.00 |
|  ...
+--------------+--------------+----------+---------------+
24 rows in set (0.03 sec)
```

7.5 Window functions, CTEs and set operations

Running total, LAG and moving average

```
SELECT month_start, claims_filed,
       SUM(claims_filed) OVER (ORDER BY month_start)                AS running_total,
       claims_filed - LAG(claims_filed) OVER (ORDER BY month_start)   AS change_vs_prev,
       ROUND(AVG(claims_filed) OVER (ORDER BY month_start
                                     ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 1) AS moving_avg_3m
  FROM v_monthly_claims
 ORDER BY month_start DESC LIMIT 12
```

Output:

```
+-------------+--------------+---------------+----------------+---------------+
| month_start | claims_filed | running_total | change_vs_prev | moving_avg_3m |
+-------------+--------------+---------------+----------------+---------------+
| 2026-11-01  |           46 |          2455 |            -55 |          80.3 |
| 2026-10-01  |          101 |          2409 |              7 |          89.7 |
| 2026-09-01  |           94 |          2308 |             20 |          82.3 |
| 2026-08-01  |           74 |          2214 |             -5 |          81.3 |
|  ...
+-------------+--------------+---------------+----------------+---------------+
12 rows in set (0.03 sec)
```

Other queries use `RANK()`, `NTILE(4)`, `ROW_NUMBER()` and `DENSE_RANK()`; a recursive CTE that walks the agent hierarchy; `INTERSECT` (370 customers hold both Health and Motor cover); and `EXCEPT` (608 Health customers have no Life cover).

8. Views, Indexes, Triggers and Transactions

8.1 Views

Views

View	Purpose
`v_policy_overview`	Policy with customer, product, agent, city, days to expiry, overdue instalments and claim count (Policies screen)
`v_claim_summary`	Claim with policy, customer, provider, adjuster, latest approved amount, payout, days to settle and **risk band** derived from the active model's threshold (Claims screen)
`v_agent_performance`	Policies sold, premium collected and commission earned per agent, with manager name
`v_monthly_claims`	Claims filed, approved, rejected and paid per month
`v_overdue_premiums`	Unpaid instalments past their due date with days overdue
`v_loss_ratio_by_type`	Premium collected vs claims paid per product
`v_payment_ledger`	Money in (premiums) and money out (payouts) as one ledger, via UNION ALL
`v_customer_directory`	Customer joined with pincode to show city and state
`v_provider_directory`	Provider joined with pincode to show city and state
`v_claim_features`	The 12 model features computed for each claim, using only information known at filing time

The risk band in `v_claim_summary` reads the threshold from the active row of `fraud_model`, so it changes automatically when a new model is registered.

8.2 Indexes

InnoDB automatically indexes every primary key, unique key and foreign key. The secondary indexes below were added for the filters and sorts used by the screens and reports. Because MySQL has no partial indexes, the status filters use composite indexes that start with the status column.

Secondary indexes

Index	Columns	Used by
`idx_claim_status_filed`	`claim(status, filed_on)`	Open-claims queue
`idx_claim_filed`	`claim(filed_on)`	Monthly trend, date ranges
`idx_policy_status_end`	`policy(status, end_date)`	Expiring-policies report, sp_expire_policies
`idx_payment_status_due`	`premium_payment(status, due_date)`	Overdue premiums, sp_lapse_overdue_policies
`idx_pincode_city`	`pincode(city)`	City filters on customers and providers
`idx_customer_name`	`customer(last_name, first_name)`	Customer search

Index	Columns	Used by
idx_document_claim	claim_document(claim_id, doc_type)	Documents per claim

The effect was measured with EXPLAIN ANALYZE on the full dataset, first without and then with the indexes:

EXPLAIN ANALYZE before and after indexing

Query	Without index	With index
Q1 Open-claims queue	Table scan on claim: 2,455 rows read, cost 248	Index range scan on idx_claim_status_filed: 102 rows, cost 46.4
Q2 Overdue premiums	Table scan on premium_payment: 10,484 rows, cost 915	Index range scan on idx_payment_status_due: 672 rows, cost 303
Q3 Policies expiring in 30 days	Table scan on policy: 3,500 rows, cost 336	Covering index range scan on idx_policy_status_end: 79 rows, cost 17.1
Q4 Customers in a city	Table scan on pincode: 450 rows, cost 105	Covering index lookup on idx_pincode_city: 30 rows, cost 43

8.3 Triggers

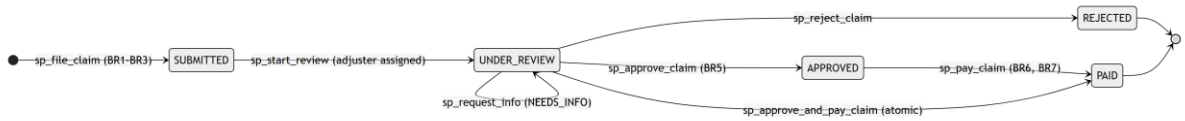
MySQL triggers have no WHEN clause and cannot modify the table that fired them, so each rule is written with IF blocks and raises an error with SIGNAL SQLSTATE '45000', which the client receives as error 1644 with a readable message. A CHECK constraint cannot read other tables, which is why cross-table rules such as BR3 must be triggers.

Triggers

Trigger	Timing / table	Rule
trg_policy_bi	BEFORE INSERT policy	Sum assured and term within the product limits
trg_claim_bi	BEFORE INSERT claim	BR1, BR2, BR3; new claims start as SUBMITTED
trg_claim_ai	AFTER INSERT claim	BR10: log the initial status
trg_claim_bu	BEFORE UPDATE claim	BR4: legal transitions, adjuster/assessment/payout guards, frozen details
trg_claim_au	AFTER UPDATE claim	BR10: log every status change
trg_assessment_bi	BEFORE INSERT claim_assessment	BR5: adjuster limit and claimed amount
trg_payout_bi	BEFORE INSERT claim_payout	BR6: claim approved, amount ≤ approved, policy cap
trg_payout_ai	AFTER INSERT claim_payout	BR7: set claim status to PAID
trg_nominee_ai / _au	AFTER INSERT / UPDATE nominee	BR8: shares ≤ 100%
trg_payment_ai / _au	AFTER INSERT / UPDATE premium_payment	BR9: re-activate a lapsed policy once nothing is overdue

8.4 Stored procedures and functions

Three functions (fn_freq_months, fn_instalments_per_year, fn_annual_premium) and 16 procedures implement the workflows. Every public procedure follows the same pattern: an exit handler that rolls back and re-raises the error, an explicit transaction, and a row lock on the claim or policy being changed. Shared steps live in core_* procedures that contain no transaction control, so they can be combined into larger atomic units. The claim life-cycle and the procedure behind each step:



Claim life-cycle

8.5 Transactions and concurrency control

Atomicity. `sp_approve_and_pay_claim` inserts an assessment, changes the claim to APPROVED and inserts a payout. If the payout violates BR6 (for example, total payouts would exceed the sum assured), the exit handler rolls back the whole transaction, so the assessment and the status change disappear as well:

```
CALL sp_approve_and_pay_claim(@claim2, 15, 300000, 'full amount', 'NEFT', @payout2, @ref2);
ERROR 1644 (45000): BR6: total payouts 550000.00 would exceed the sum assured 500000.00


SELECT claim_id, status FROM claim WHERE claim_id = @claim2;            -- 2457 | UNDER_REVIEW
SELECT COUNT(*) AS assessments_left_behind FROM claim_assessment
 WHERE claim_id = @claim2;                                             -- 0
```

Savepoints. The demo script also uses `SAVEPOINT after_spouse` and `ROLLBACK TO SAVEPOINT` to undo one wrong nominee row without losing the earlier one in the same transaction.

Isolation and locking. InnoDB runs at REPEATABLE READ. `core_lock_claim` reads the claim with `SELECT … FOR UPDATE`, which places an exclusive row lock. A second session trying to act on the same claim waits until the first commits, then sees the new status and fails cleanly. `scripts/concurrency_demo.py` runs two real database sessions in parallel threads:

```
PS D:\VSCODE\projects\DBMS Project '26> python scripts/concurrency_demo.py
  [  0.11s] setup      --- Demo A: two sessions approve claim 2461 at the same time ---
  [  0.18s] session1   locked claim, status = UNDER_REVIEW
  [  0.19s] session1   approved inside the open transaction (not committed yet)
  [  0.19s] session2   CALL sp_approve_claim(...) -> waits for the row lock
  [  2.19s] session1   COMMIT (lock released)
  [  2.19s] session2   blocked 2.00s, then failed: errno 1644: Claim 2461 is APPROVED, expected UNDER_REVIEW
  [  2.19s] check      APPROVED assessments on the claim: 1 (exactly one)
  [  2.19s] setup      --- Demo B: deadlock on claims 2462 and 2463 ---
  [  2.27s] session1   locked claim 2462
  [  2.27s] session2   locked claim 2463
  [  2.77s] session1   requesting lock on claim 2463
  [  2.77s] session2   requesting lock on claim 2462
  [  2.77s] session1   got both locks, COMMIT
  [  2.77s] session2   ROLLED BACK by InnoDB: errno 1213: Deadlock found when trying to get lock; try restarting transaction
  [  2.81s] cleanup    removed 3 demo claims
PS D:\VSCODE\projects\DBMS Project '26>
```

Concurrency demo: row lock and deadlock

Demo A shows that only one of two simultaneous approvals succeeds. Demo B makes two sessions lock two claims in opposite order; InnoDB detects the deadlock, rolls back one session (error 1213) and lets the other finish. The API maps error 1213 to HTTP 409 so the user can simply retry.

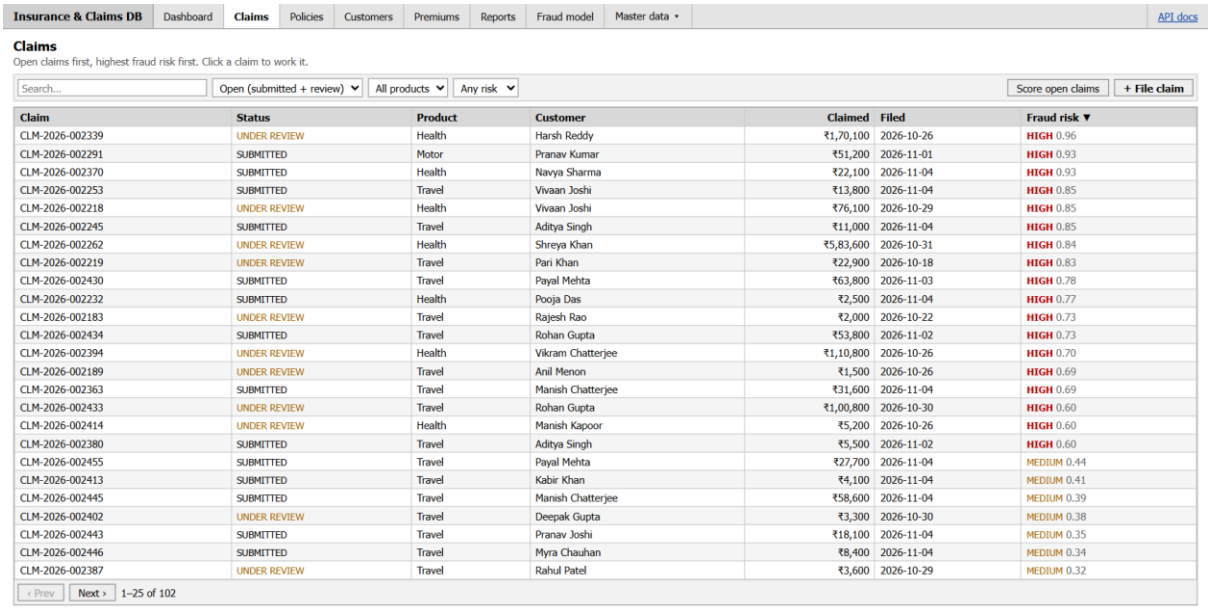
9. Front-End Design and Database Connectivity

9.1 Front-end design

The front end is a single-page application written in plain HTML, CSS and JavaScript, served by the API at `http://127.0.0.1:8000`. It was deliberately kept plain, in the style of database administration tools such as phpMyAdmin: a top menu bar, grey tables with sortable column headers, search boxes, filter drop-downs and pagination.

Screens

Menu	What it shows / does
Dashboard	Key figures (active policies, open claims, high-risk claims, 12-month premium and payouts, loss ratio, overdue instalments) and four bar charts
Claims	Work queue of open claims sorted by fraud risk; filters by status, product and risk band; "File claim"; clicking a claim opens its detail panel with the workflow buttons, documents, assessments and audit trail
Policies	All policies with filters; "Sell policy"; policy detail with nominees, instalments and claims
Customers	Generic CRUD screen (list, search, add, edit, delete)
Premiums	Overdue instalments with a Pay button, plus batch actions (generate instalments, lapse overdue, expire policies)
Reports	Eight reports as tables with CSV download
Fraud model	Active model, threshold, test metrics, model comparison, confusion matrix and charts
Master data	CRUD screens for agents, products, providers, adjusters and pincodes



Claims work queue, riskiest first

9.2 Database connectivity

The browser never talks to MySQL directly. It calls the REST API with `fetch()`, and the FastAPI application talks to MySQL through **mysql-connector-python** with a **connection pool**, so each request borrows an open connection instead of opening a new one. Connection settings are read from a `.env` file, and the application uses its own MySQL account (`insurance_app`) with rights only on the project databases.

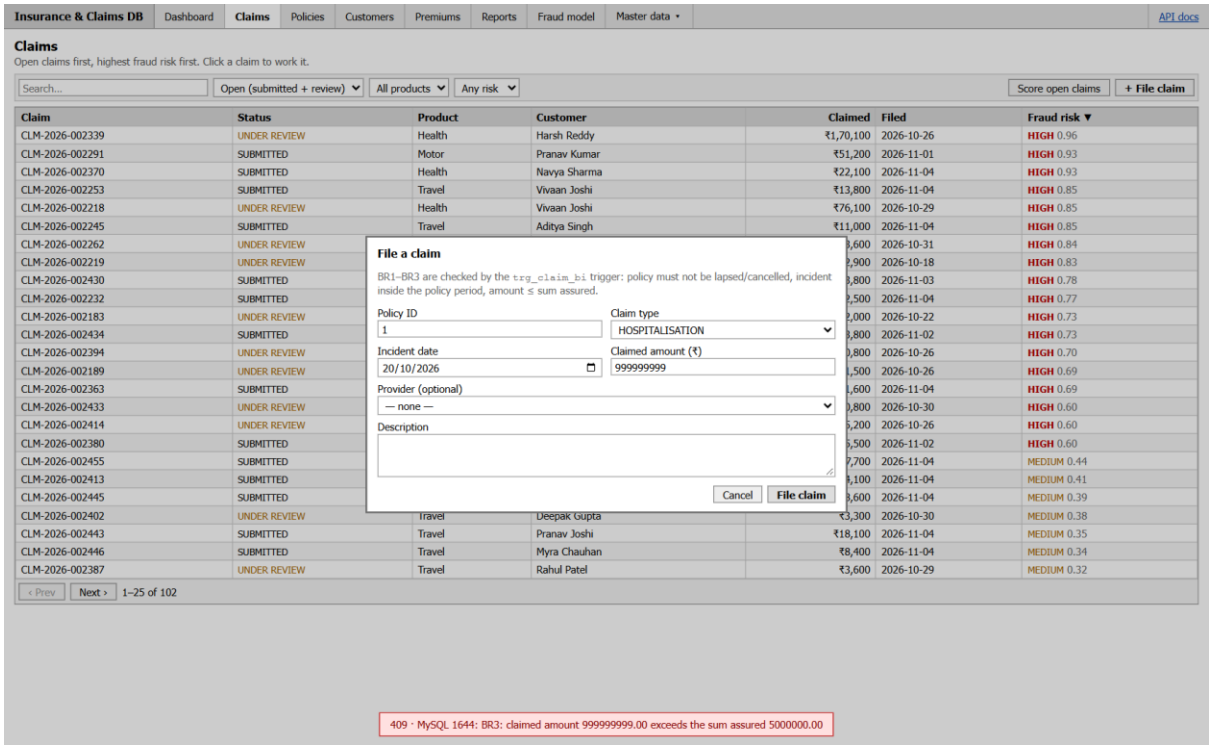
```
# api/db.py (excerpt)
def get_pool():
    """One pool per process. autocommit=True: the stored procedures own their transactions."""
    global _pool
    if _pool is None:
        _pool = pooling.MySQLConnectionPool(
            pool_name="insurance_pool", pool_size=int(ENV.get("DB_POOL_SIZE", 10)), pool_reset_session=True,
            host=ENV.get("DB_HOST", "127.0.0.1"), port=int(ENV.get("DB_PORT", 3306)),
```

```
            user=ENV["DB_USER"], password=ENV["DB_PASSWORD"],
            database=os.environ.get("DB_NAME", ENV.get("DB_NAME")),
            autocommit=True, charset="utf8mb4", collation="utf8mb4_0900_ai_ci")
    return _pool
```

The workflow endpoints only call stored procedures (for example `POST /api/claims/{id}/approve-and-pay` calls `sp_approve_and_pay_claim`), so the transaction logic stays in the database. When MySQL rejects an operation, the API returns its message with a suitable HTTP status, and the front end shows it in a red message bar:

Mapping of MySQL errors to HTTP responses

MySQL error	Meaning	HTTP status
1644	Business rule raised by a trigger or procedure (SIGNAL)	409 Conflict
1062	Duplicate value for a UNIQUE key	409 Conflict
1451 / 1452	Foreign key: parent in use / parent missing	409 Conflict
1213	Deadlock, transaction rolled back	409 Conflict
3819	CHECK constraint violated	422 Unprocessable
1048, 1366, 1292, 1406, 1364, 1265	NULL / wrong type / bad date / too long / missing value	422 Unprocessable



A claim above the sum assured, rejected by the BR3 trigger

9.3 Security

- **SQL injection:** every value is sent as a bound parameter (`%s`). Table names, column names and sort columns can only come from a fixed whitelist in `api/crud.py`; anything else is rejected with HTTP 400 or 422. Automated tests send inputs such as `' OR '1'='1` and `email; DROP TABLE customer` to confirm this.
- **Least privilege:** the app connects as `insurance_app`, not as root.

- **Read-only paths:** premium instalments cannot be edited through generic CRUD (HTTP 405); they change only through `sp_pay_premium` and the batch procedures.

10. CRUD Operations and Report Generation

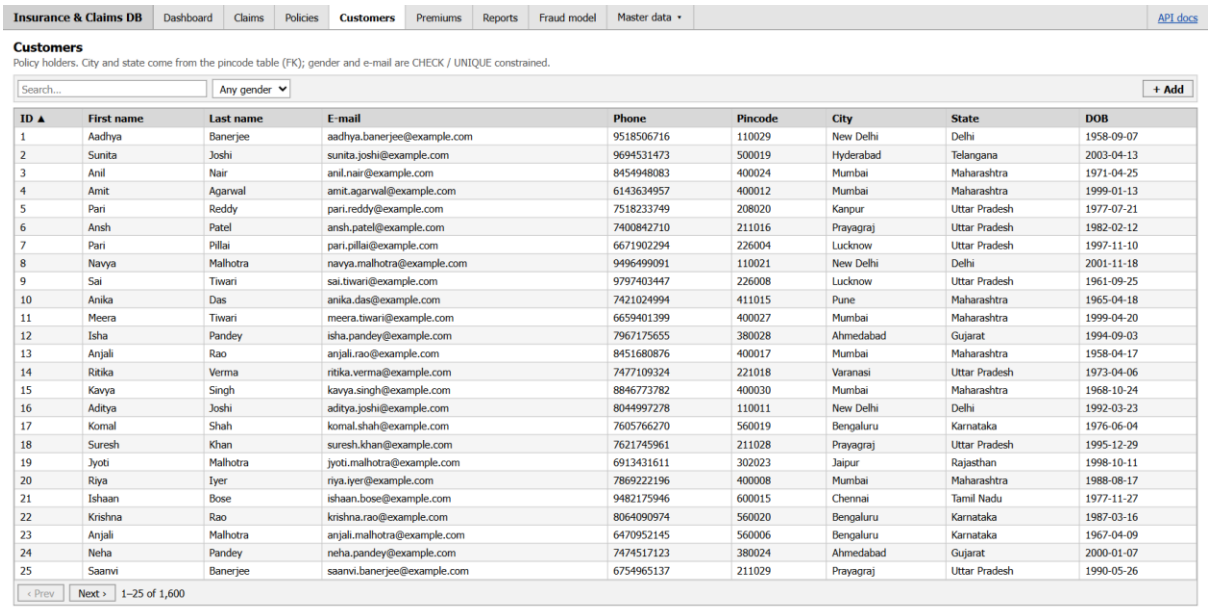
10.1 Generic CRUD

Instead of writing separate code for every master table, one generic CRUD module serves all of them. Each resource is described once in a whitelist: its table, primary key, writable columns, searchable columns, filters and (optionally) a view to read from. For customers and providers the view adds the city and state from the pincode table, while writes still go to the base table.

Generic CRUD endpoints

HTTP request	SQL executed	Notes
`GET /api/{resource}?search=&sort=&order=&limit=&offset=&<filter>=`	`SELECT … WHERE … LIKE %s … ORDER BY … LIMIT %s OFFSET %s, plus a COUNT(*)`	Search, filter, sort, paginate
`GET /api/{resource}/{id}`	`SELECT * … WHERE pk = %s`	404 if missing
`POST /api/{resource}`	`INSERT INTO … VALUES (%s, …)`	201 + the new row
`PUT /api/{resource}/{id}`	`UPDATE … SET col = %s … WHERE pk = %s`	Partial update
`DELETE /api/{resource}/{id}`	`DELETE FROM … WHERE pk = %s`	409 if the row is still referenced

Resources covered: pincodes, customers, agents, policy types, providers, adjusters, nominees, documents (full CRUD) and payments (read-only). Policies and claims are created through their procedures and listed through their views.

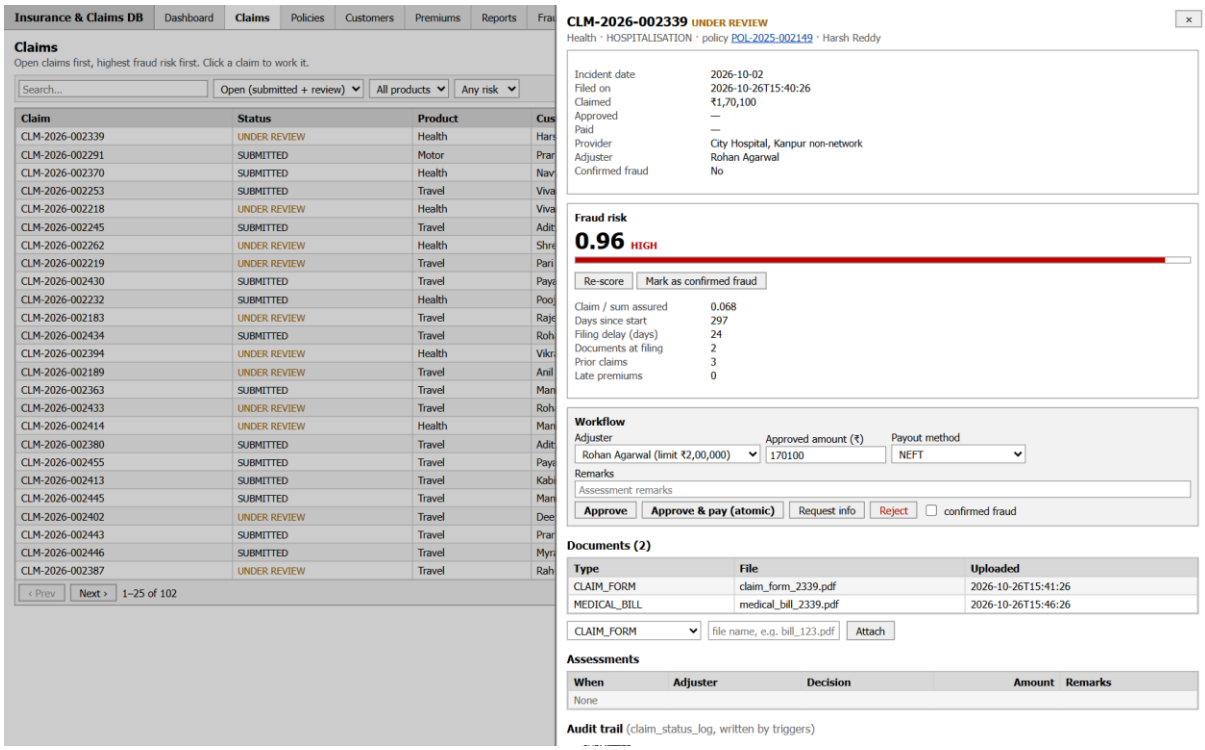


Customers screen

10.2 Workflow operations

Operations that touch several tables are not generic CRUD; each one is a dedicated endpoint that calls one stored procedure:

- `POST /api/policies` → `sp_sell_policy` (computes premium, number, first instalment).
- `POST /api/claims` → `sp_file_claim`; then `start-review`, `request-info`, `approve`, `reject`, `pay`, `approve-and-pay` → the matching `sp_*` procedure.
- `POST /api/payments/{id}/pay` → `sp_pay_premium`.
- `POST /api/admin/generate-instalments`, `/lapse-overdue`, `/expire-policies` → the batch procedures.



Claim detail with fraud risk and workflow actions

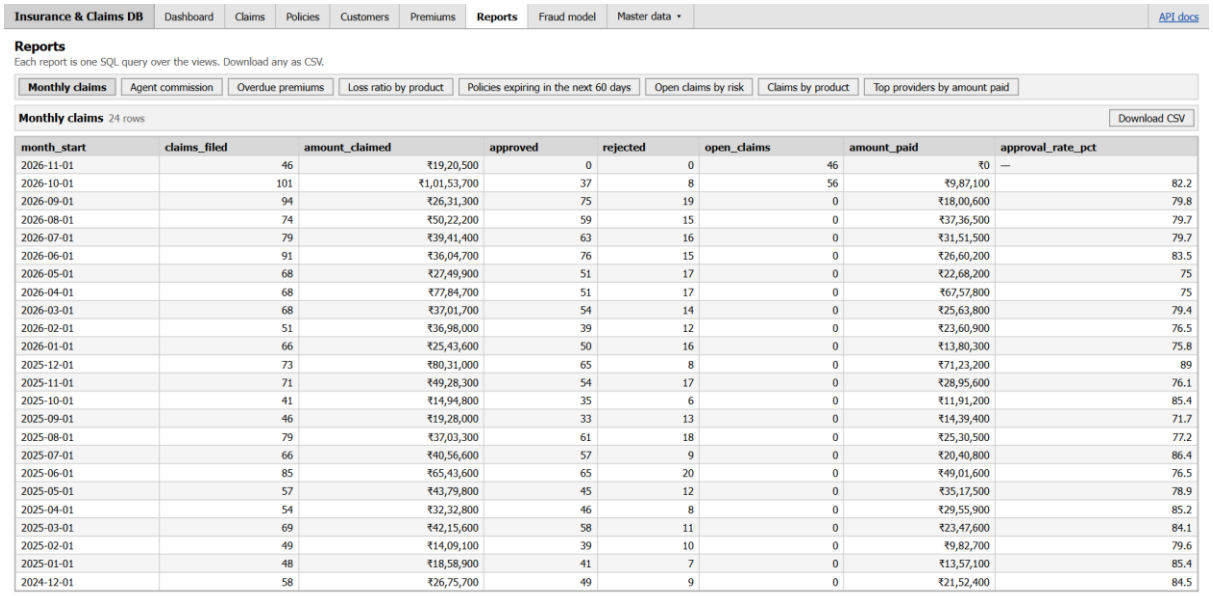
10.3 Reports

Each report is a single SQL query over the views and can be viewed on screen or downloaded as CSV (`?format=csv`).

Reports

Report	Source	Business question
Monthly claims	`v_monthly_claims`	How many claims are filed, approved, rejected and paid each month?
Agent commission	`v_agent_performance`	How much premium did each agent bring in, and what commission is due?
Overdue premiums	`v_overdue_premiums`	Which instalments are late, and by how many days?
Loss ratio by product	`v_loss_ratio_by_type`	For each product, what share of premium is paid out as claims?
Policies expiring in 60 days	`v_policy_overview`	Which policies should be offered for renewal?
Open claims by risk	`v_claim_summary`	Which open claims should adjusters review first?

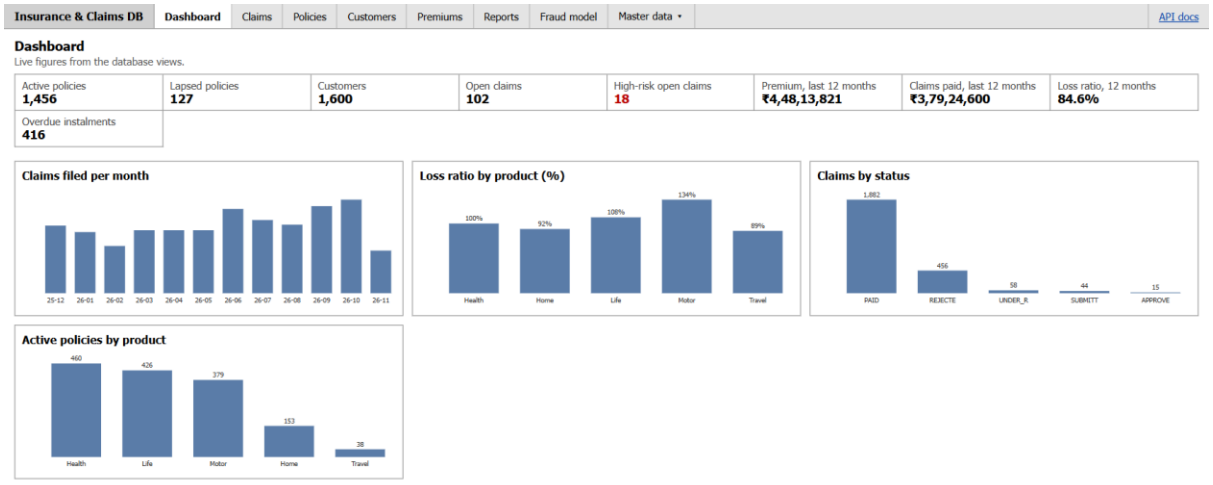
Report	Source	Business question
Claims by product	`v_claim_summary`	Claim count, amounts and average days to settle per product
Top providers by amount paid	`v_claim_summary`	Which hospitals and garages receive the most money?



month_start	claims_filed	amount_claimed	approved	rejected	open_claims	amount_paid	approval_rate_pct
2026-11-01	46	₹19,20,500	0	0	46	₹0	—
2026-10-01	101	₹1,01,53,700	37	8	56	₹9,87,100	82.2
2026-09-01	94	₹26,31,300	75	19	0	₹18,00,600	79.8
2026-08-01	74	₹50,22,200	59	15	0	₹37,36,500	79.7
2026-07-01	79	₹39,41,400	63	16	0	₹31,51,500	79.7
2026-06-01	91	₹36,04,700	76	15	0	₹26,60,200	83.5
2026-05-01	68	₹27,49,900	51	17	0	₹22,68,200	75
2026-04-01	68	₹77,84,700	51	17	0	₹67,57,800	75
2026-03-01	68	₹37,01,700	54	14	0	₹25,63,800	79.4
2026-02-01	51	₹36,98,000	39	12	0	₹23,60,900	76.5
2026-01-01	66	₹25,43,600	50	16	0	₹13,80,300	75.8
2025-12-01	73	₹80,31,000	65	8	0	₹71,23,200	89
2025-11-01	71	₹49,28,300	54	17	0	₹28,95,600	76.1
2025-10-01	41	₹14,94,800	35	6	0	₹11,91,200	85.4
2025-09-01	46	₹19,28,000	33	13	0	₹14,39,400	71.7
2025-08-01	79	₹37,03,300	61	18	0	₹25,30,500	77.2
2025-07-01	66	₹40,56,600	57	9	0	₹20,40,800	86.4
2025-06-01	85	₹65,43,600	65	20	0	₹49,01,600	76.5
2025-05-01	57	₹43,79,800	45	12	0	₹35,17,500	78.9
2025-04-01	54	₹32,32,800	46	8	0	₹29,55,900	85.2
2025-03-01	69	₹42,15,600	58	11	0	₹23,47,600	84.1
2025-02-01	49	₹14,09,100	39	10	0	₹9,82,700	79.6
2025-01-01	48	₹18,58,900	41	7	0	₹13,57,100	85.4
2024-12-01	58	₹26,75,700	49	9	0	₹21,52,400	84.5

Reports screen

Example result, loss ratio by product (claims paid ÷ premium collected): Health 99.53%, Life 108.35%, Motor 133.60%, Home 91.69%, Travel 89.02%. A loss ratio above 100% means the product paid out more than it collected; in this dataset that is the case for Motor and Life.



Dashboard

11. AI/ML Implementation and Model Evaluation

11.1 Problem definition

The task is **binary classification**: given a claim at the moment it is filed, estimate the probability that it is fraudulent. The label is `claim.fraud_flag`, which an adjuster sets when fraud is confirmed

(typically on rejection, or later after payment). Only **resolved** claims (REJECTED or PAID) are used for training, because open claims have no final label yet.

The output is used to **rank** the work queue and to show a LOW / MEDIUM / HIGH band. It never approves or rejects a claim automatically; the decision stays with the adjuster.

11.2 Data and features

Training data comes straight from the database through the view `v_claim_features`. 2,353 resolved claims were available, of which 210 (8.9%) are confirmed fraud, so the classes are imbalanced. Features use **only information known when the claim is filed**; for example, the document count includes only documents uploaded within one day of filing. This prevents *data leakage*: using later information such as the payout would make the model look good in testing but useless on new claims.

Model features (computed in v_claim_features)

Feature	Meaning
`claimed_amount`	Amount claimed
`claim_ratio`	Claimed amount ÷ sum assured
`days_since_start`	Days between policy start and the incident
`days_to_end`	Days between the incident and policy end
`filing_delay`	Days between the incident and filing
`customer_age`	Age of the policy holder at the incident
`prior_claims`	Number of earlier claims by the same customer
`late_payments`	Number of premium instalments that were paid late or missed
`documents_attached`	Documents uploaded within one day of filing
`non_network_provider`	1 if the provider is outside the insurer's network
`claim_type`	Category (one-hot encoded)
`policy_type`	Product (one-hot encoded)

11.3 Training method

1. Stratified split: 80% training (1,882 claims) and 20% held-out test (471 claims). The test set is used only once, at the very end.
2. Three candidate models in scikit-learn pipelines (scaling and one-hot encoding inside the pipeline): **Logistic Regression**, **Random Forest** and **Histogram Gradient Boosting**, each with class weighting for the imbalance.
3. Model selection by **5-fold stratified cross-validation ROC-AUC** on the training set.
4. The decision **threshold** is chosen to maximise F1 on out-of-fold predictions of the training set, not on the test set.
5. The selected model is refitted on the full training set, evaluated once on the test set, saved with joblib, and **registered in the database** with `CALL sp_register_model(…)`, which deactivates the old model and activates the new one in a single transaction.

Cross-validation results (training set)

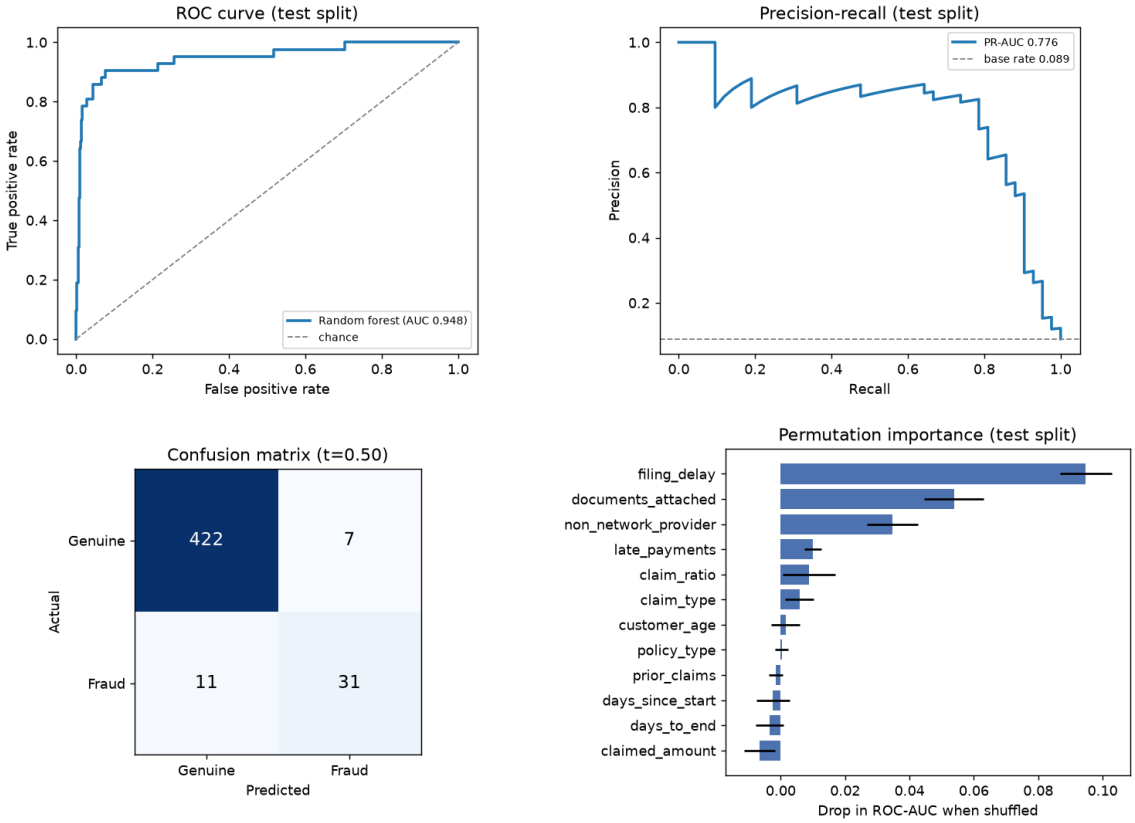
Model	Mean ROC-AUC	Std. dev.
Logistic regression	0.9089	0.0196
Random forest (selected)	0.9329	0.0095
Hist. gradient boosting	0.9135	0.0151

11.4 Evaluation on the held-out test set

Test-set metrics of the selected model

Metric	Value	What it means
ROC-AUC	0.948	Probability that a random fraud claim is scored higher than a random genuine one
PR-AUC (average precision)	0.776	Area under the precision-recall curve (a random model would score about 0.09)
Precision	0.816	Of the claims flagged HIGH, the share that really were fraud
Recall	0.738	Of all fraud claims, the share that were flagged
F1 score	0.775	Harmonic mean of precision and recall
Threshold	0.505	Score at or above which a claim is HIGH risk

Confusion matrix on the 471 test claims: **31** frauds correctly flagged (true positives), **11** frauds missed (false negatives), **7** genuine claims wrongly flagged (false positives) and **422** genuine claims correctly passed (true negatives).



Model evaluation on the test set

Permutation importance shows that the model relies mainly on **filing delay**, **number of documents** and **non-network provider**, followed by late premium payments and the claim-to-cover ratio. These match the warning signs that claim investigators look for in practice.

11.5 Deployment in the application

- When a claim is filed, and again whenever a document is attached to an open claim, the API reads the claim's features from `v_claim_features`, scores it and writes `claim.fraud_score`.
- A "Score open claims" button re-scores all open claims in one batch.

- `v_claim_summary` turns the score into a risk band using the threshold of the active model in `fraud_model`; the Claims screen sorts by this score.
- The Fraud model screen shows the registered model, its metrics and these charts.

11.6 Limitations

The dataset is synthetic. It was generated with a deliberate, realistic fraud signal so that the pipeline could be built and tested end to end. The metrics therefore show that the pipeline works correctly, not how accurate the model would be on a real insurer's claims. With real data, the same code would simply be retrained, and the threshold would be re-tuned to the investigation team's capacity.

12. Testing, Results and Screenshots

12.1 Testing approach

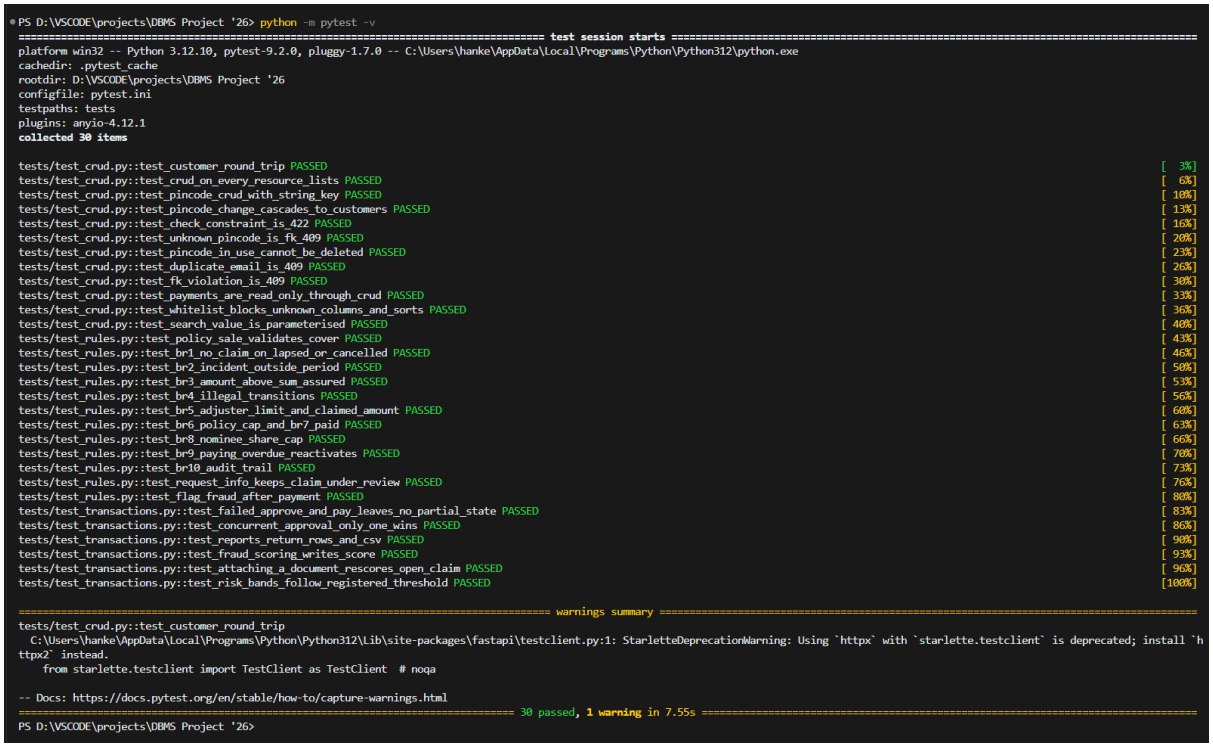
- **Automated tests (pytest):** 30 tests call the real API, which talks to a separate `insurance_test` database built from the same SQL scripts. They cover CRUD, every business rule, atomicity, concurrency, reports and ML scoring.
- **Rule demonstrations:** `db/08_rule_demos.sql` performs each forbidden action on purpose and records the database's error. It cleans up after itself.
- **Performance:** `EXPLAIN ANALYZE` before and after indexing.
- **Concurrency:** two real sessions in parallel threads.
- **Manual UI testing** of every screen in Chrome.

All evidence is regenerated with one command, `python scripts/run_everything.py`, and saved under `docs/outputs/`.

12.2 Automated test results

Automated test suite

Test file	Tests	What is checked	Result
`test_crud.py`	12	CRUD round trips for every resource; search, filter, sort, pagination; ON UPDATE CASCADE of pincode; CHECK → 422; FK and duplicate key → 409; payments read-only (405); column whitelist and parameterised search (SQL injection attempts fail)	12 / 12 pass
`test_rules.py`	12	Policy cover validation; each of BR1–BR10; request-info keeps the claim under review; fraud can be confirmed after payment	12 / 12 pass
`test_transactions.py`	6	Failed approve-and-pay leaves no partial state; four concurrent approvals → exactly one succeeds; all reports and CSV export; fraud scoring, re-scoring on document upload, risk bands follow the registered threshold	6 / 6 pass



Test run: all 30 tests pass

12.3 Business-rule demonstrations

Each of the following operations was attempted on the real database and was correctly rejected. The error numbers are MySQL's: 1644 = rule raised by a trigger, 3819 = CHECK, 1451/1452 = foreign key, 1062 = duplicate key.

Expected errors from 08_rule_demos.sql

#	Attempted operation	Stopped by	Error
1	Sum assured outside product range	trg_policy_bi	1644
2	Pincode not 6 digits	CHECK ck_pincode_format	3819
3	Instalment PAID without paid_on	CHECK ck_payment_paid_on	3819
4	Customer with unknown pincode	FK fk_customer_pincode	1452
5	Incident before policy start	BR2 (trg_claim_bi)	1644
6	Claim larger than sum assured	BR3 (trg_claim_bi)	1644
7	SUBMITTED → APPROVED directly	BR4 (trg_claim_bu)	1644
8	Review without an adjuster	BR4 (trg_claim_bu)	1644
9	Changing amount after review started	trg_claim_bu	1644
10	Approval above adjuster limit	BR5 (trg_assessment_bi)	1644
11	Approval above claimed amount	BR5 (trg_assessment_bi)	1644
12	Paying a claim that is not approved	BR6 (trg_payout_bi)	1644
13	Payout above approved amount	BR6 (trg_payout_bi)	1644
14	Approve-and-pay above remaining cover (rolled back)	BR6 + transaction	1644
15	Nominee shares 110% (insert)	BR8 (trg_nominee_ai)	1644
16	Nominee shares 120% (update)	BR8 (trg_nominee_au)	1644
17	Claim on a LAPSED policy	BR1 (trg_claim_bi)	1644
18	Claim on a CANCELLED policy	BR1 (trg_claim_bi)	1644
19	Document for a missing claim	FK fk_doc_claim	1452

#	Attempted operation	Stopped by	Error
20	Deleting a product in use	FK fk_policy_type (RESTRICT)	1451
21	Duplicate customer e-mail	UNIQUE uq_customer_email	1062

The same script also shows the successful paths: BR7 (payout sets the claim to PAID), BR9 (a lapsed policy becomes ACTIVE again when the overdue instalment is paid), BR10 (audit rows for each change), a savepoint rollback, and the instalment-generation, lapse and expiry batch procedures.

12.4 Results summary

Final state of the loaded database

Item	Value
Customers / agents / providers / adjusters	1,600 / 40 / 120 / 15
Policies (active / expired / lapsed / cancelled)	3,500 (active 1,456, cancelled 40, expired 1,877, lapsed 127)
Premium instalments	10,484
Claims (by status)	2,455 (APPROVED 15, PAID 1,882, REJECTED 456, SUBMITTED 44, UNDER_REVIEW 58)
Documents / assessments / payouts / audit rows	7,252 / 2,484 / 1,882 / 9,101
Automated tests	30 / 30 passed
Rule demonstrations	21 / 21 rejected as expected
Fraud model (test set)	Random forest: ROC-AUC 0.948, precision 0.82, recall 0.74

13. Conclusion and Future Scope

13.1 Conclusion

We designed and built a complete Insurance & Claims Management System on MySQL 8. Starting from an unnormalized policy register, we arrived at 15 tables in BCNF, linked by 16 foreign keys with chosen referential actions. All ten business rules are enforced inside the database by constraints and triggers, so the data stays consistent no matter which client writes to it. The policy and claim life-cycles are implemented as transactional stored procedures, and we showed that they stay correct under failure (full rollback of a partly completed approve-and-pay) and under concurrent use (row locking and deadlock detection).

The project uses the full range of SQL covered in the course, including joins of every kind, correlated subqueries, ROLLUP, window functions, recursive CTEs and set operations. Four of the seven secondary indexes were measured with `EXPLAIN ANALYZE`, and each turned a full table scan into an index scan; the other three support customer search, the monthly claims trend and document lookups. A FastAPI back end and a plain web interface make the database usable by non-technical staff through CRUD screens, a claims work queue and eight reports. A Random Forest model, trained only on information available at filing time, ranks open claims by fraud risk with a test ROC-AUC of 0.948, and its threshold is managed through the database itself. Thirty automated tests and twenty-one rule demonstrations confirm that the system behaves as specified.

13.2 Future scope

- **Authentication and roles:** separate logins for servicing staff, adjusters and managers, with MySQL roles and API permissions (for example, only managers can register a model or cancel a policy).
- **Document storage:** upload the actual files (to object storage) and extract data from them with OCR.
- **Customer portal:** let customers view policies, pay premiums and file claims online, with SMS/e-mail notifications on status changes.
- **Real data and model monitoring:** retrain on real claims, monitor drift, and explain each score to the adjuster (e.g. SHAP values).
- **Scale:** partition the claim and payment tables by year, add read replicas for reporting, and archive old audit rows.
- **More products and rules:** co-payments, deductibles, waiting periods, policy renewals and endorsements.

14. References

1. A. Silberschatz, H. F. Korth and S. Sudarshan, *Database System Concepts*, 7th ed., McGraw-Hill, 2019.
2. R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed., Pearson, 2016.
3. Oracle Corporation, *MySQL 8.0 Reference Manual*, https://dev.mysql.com/doc/refman/8.0/en/ (triggers, stored programs, InnoDB locking, EXPLAIN ANALYZE, window functions, CTEs).
4. Oracle Corporation, *MySQL Connector/Python Developer Guide*, https://dev.mysql.com/doc/connector-python/en/.
5. S. Ramírez, *FastAPI documentation*, https://fastapi.tiangolo.com/.
6. F. Pedregosa et al., "Scikit-learn: Machine Learning in Python", *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
7. L. Breiman, "Random Forests", *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
8. T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets", *PLOS ONE*, 10(3), 2015.
9. pytest documentation, https://docs.pytest.org/.